

SE3082 – Parallel Computing

Exercise 5



IT24103470
Kathisan A

Exercise 5 Details, Code & Output Verification

1. Exercise Task Description

Parallelize the serial recursive code for Fibonacci series using OpenMP task parallelization directives.

2. Complete C Source Code (exercise5.c)

```
#include <stdio.h>
#include <omp.h>

int fib(int n) {
    int i, j;
    if (n < 2) return n;
    if (n < 20) return fib(n - 1) + fib(n - 2);

    #pragma omp task shared(i) firstprivate(n)
    i = fib(n - 1);

    #pragma omp task shared(j) firstprivate(n)
    j = fib(n - 2);

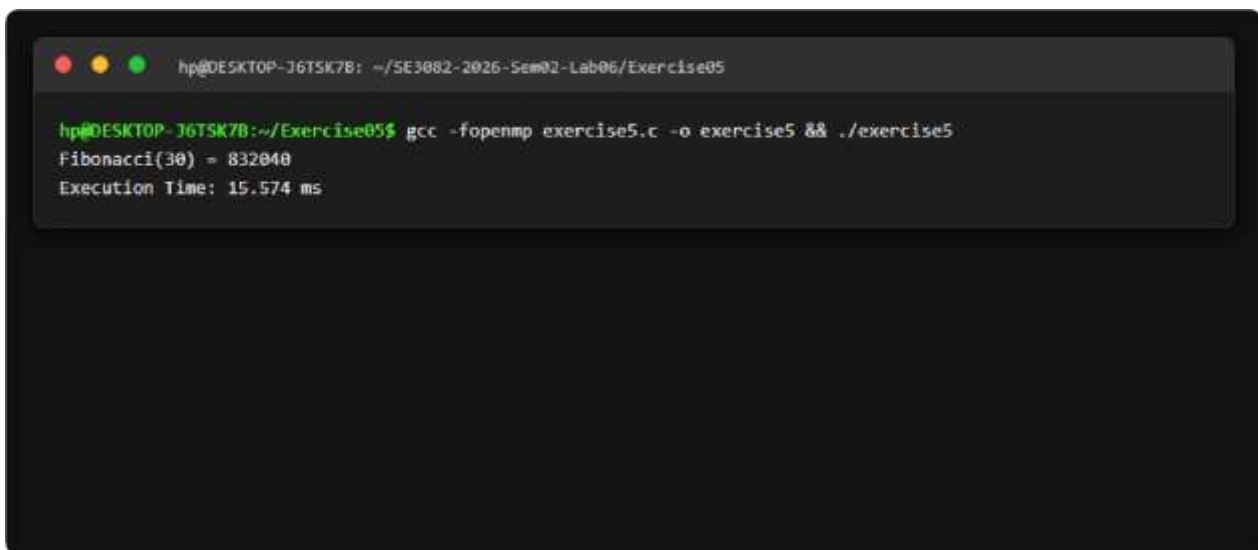
    #pragma omp taskwait
    return i + j;
}

int main() {
    int n = 30, result;
    double tstart = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        { result = fib(n); }
    }

    double tstop = omp_get_wtime();
    printf("Fibonacci(%d) = %d | Time: %.3f ms\n", n, result, (tstop - tstart) *
1000.0);
    return 0;
}
```

3. Terminal Execution Output Screenshot



```
hp@DESKTOP-J6TSK7B: ~/5E3882-2026-Sem02-Lab06/Exercise05
hp@DESKTOP-J6TSK7B:~/Exercise05$ gcc -fopenmp exercise5.c -o exercise5 && ./exercise5
Fibonacci(30) = 832040
Execution Time: 15.574 ms
```

4. Directive Analysis & Explanation

How Task Parallelization works:

Recursive subproblems $\text{fib}(n-1)$ and $\text{fib}(n-2)$ are spawned as explicit asynchronous OpenMP tasks using `#pragma omp task`. The `#pragma omp task wait` directive forces the parent task to wait until both child tasks finish before adding their results.